

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



BÁO CÁO THỰC TẬP

NGÀNH: CÔNG NGHỆ THÔNG TIN

ĐỀ TÀI: XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG
QUẢN LÝ TIẾN ĐỘ ĐỌC, CHIA SẺ VÀ ĐỀ XUẤT
SÁCH ỨNG DỤNG MLOPS VÀ DEVSECOPS

Cán bộ hướng dẫn: Nguyễn Quốc Thắng

Giảng viên đánh giá: ThS. Ngô Xuân
Trường

Sinh viên: Nguyễn Quang Dũng

Mã sinh viên: 23021497

Lớp: K68-CS3

Hà Nội, tháng 10 năm 2026

MỤC LỤC

LỜI CẢM ƠN	ii
I. GIỚI THIỆU CHUNG	5
II. YÊU CẦU BÀI TOÁN	6
III. LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN VÀ QUÁ TRÌNH PHÁT TRIỂN HỆ THỐNG	8
IV. CÁCH CÀI ĐẶT PHẦN MỀM	13
V. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN TIẾP THEO	14
TÀI LIỆU THAM KHẢO	21

LỜI CẢM ƠN

Trong quá trình thực tập và thực hiện báo cáo với đề tài này, em đã nhận được sự quan tâm, hướng dẫn và hỗ trợ quý báu từ nhà trường, đơn vị thực tập cùng các thầy cô, anh chị hướng dẫn.

Trước hết, em xin chân thành cảm ơn Ban lãnh đạo và các anh chị tại JITS Innovation Labs đã tạo điều kiện để em được tiếp cận môi trường làm việc thực tế, củng cố kiến thức chuyên môn và rèn luyện kỹ năng nghề nghiệp. Đặc biệt, em xin bày tỏ lòng biết ơn sâu sắc tới cán bộ hướng dẫn Nguyễn Quốc Thắng đã tận tình định hướng, góp ý và hỗ trợ em trong suốt quá trình thực hiện đề tài.

Em cũng xin trân trọng cảm ơn các thầy cô Trường Đại học Công nghệ, Đại học Quốc gia Hà Nội đã trang bị cho em nền tảng kiến thức cần thiết. Em xin cảm ơn ThS. Ngô Xuân Trường đã dành thời gian đánh giá và góp ý cho báo cáo.

Mặc dù đã nỗ lực hoàn thiện, báo cáo khó tránh khỏi những thiếu sót do kinh nghiệm thực tế còn hạn chế. Em rất mong nhận được những ý kiến đóng góp để tiếp tục hoàn thiện kiến thức và kỹ năng trong thời gian tới.

Em xin trân trọng cảm ơn.

I. GIỚI THIỆU CHUNG

a. Giới thiệu Công ty TNHH JITS Innovation Labs

Công ty TNHH JITS Innovation Labs là doanh nghiệp hoạt động trong lĩnh vực phát triển phần mềm và cung cấp dịch vụ công nghệ thông tin tại Hà Nội. Công ty bắt đầu hoạt động từ tháng 6 năm 2016, có quy mô từ 11 đến 50 nhân sự, đặt trụ sở tại Hà Nội. Quy mô này tạo điều kiện để các nhóm kỹ thuật phối hợp trực tiếp, đồng thời yêu cầu mỗi thành viên chủ động trong quá trình nghiên cứu, phát triển và triển khai sản phẩm.

Lĩnh vực chuyên môn của JITS Innovation Labs là các giải pháp công nghệ cho ngân hàng và tổ chức tài chính, bao gồm nền tảng ngân hàng số, hệ thống điểm bán hàng, xử lý dữ liệu theo luồng và tích hợp hệ thống. Đội ngũ của công ty có kinh nghiệm triển khai, tùy biến hệ thống ngân hàng lõi cho khách hàng doanh nghiệp, hỗ trợ các ngân hàng và tổ chức tài chính vì mô cải tiến sản phẩm và thực hiện chuyển đổi số. Đặc thù của các hệ thống tài chính đòi hỏi phần mềm có tính ổn định, khả năng mở rộng, bảo mật và giám sát tốt; vì vậy, các phương pháp DevOps và DevSecOps có vai trò thiết thực trong hoạt động phát triển và vận hành sản phẩm.

Bên cạnh hoạt động phát triển sản phẩm, doanh nghiệp tổ chức chương trình thực tập theo hướng đào tạo gắn với thực hành. Tham gia vào kỳ thực tập năm nay, sinh viên chúng em được tiếp cận kiến thức theo lộ trình, hoàn thành các bài tập nhỏ cho từng chủ đề, lưu trữ sản phẩm trên hệ thống quản lý mã nguồn và trình bày kết quả định kỳ. Hình thức này giúp chúng em hình thành thói quen tự học, viết tài liệu, kiểm chứng kết quả và chịu trách nhiệm đối với sản phẩm đã thực hiện. Trong kỳ thực tập, em tham gia chương trình đào tạo DevOps và lựa chọn hướng chuyên sâu MLOps và Security để vận dụng kiến thức vào dự án tổng hợp.[1, 2, 3].

b. Giới thiệu công việc thực tập

Trong thời gian thực tập tại JITS Innovation Labs, em đảm nhiệm vị trí DevOps Intern. Em vận dụng kiến thức về Linux, Git, Docker, CI/CD, Kubernetes, Terraform, MLOps và DevSecOps để hoàn thành các bài thực hành và dự án tổng hợp. Quá trình này giúp em hình thành cách tiếp cận công việc từ tổ chức mã nguồn, xây dựng môi trường chạy đến kiểm tra và vận hành sản phẩm.

Từ các kiến thức nền tảng, em xây dựng và đóng gói các ứng dụng web nhiều thành phần, triển khai trên Kubernetes, quản lý hạ tầng bằng Terraform theo mô hình Infrastructure as Code (IaC), kết hợp với kiểm thử tự động và cơ chế CI/CD. Em cũng ứng dụng thêm MLOps và bảo mật vào việc pipeline xử lý dữ liệu, huấn luyện, đánh giá và triển khai mô hình.

Những kinh nghiệm trên được em ứng dụng để phát triển dự án cá nhân Philobiblus, một ứng dụng web hỗ trợ quản lý tiến độ đọc, nhận xét, chia sẻ và gợi ý sách. Sản phẩm kết hợp các nội dung đã thực hành để tổ chức mã nguồn, đóng gói ứng dụng, tự động kiểm thử và triển khai thử nghiệm. Qua đó, em có thể hoàn thiện một sản phẩm từ giai đoạn phát triển đến môi trường vận hành và đánh giá được các yêu cầu về tính ổn định, khả năng mở rộng và bảo mật.

c. Giới thiệu bài toán

Trọng tâm của bài toán là kiểm chứng cách tổ chức và vận hành một ứng dụng web nhiều thành

phần: hệ thống phải ổn định khi frontend, backend, cơ sở dữ liệu và model học máy hoạt động đồng thời; bảo vệ dữ liệu theo quyền sở hữu và phạm vi truy cập; cô lập và khôi phục thành phần gặp lỗi; có khả năng autoscale khi tải và số người dùng đồng thời biến động. Philobiblus được em phát triển như là một sản phẩm mẫu để kiểm chứng các yêu cầu trên, có thể hỗ trợ quản lý thư viện và hoạt động đọc, gồm cập nhật tiến độ, nhận xét, chia sẻ và gợi ý sách. Hệ thống được triển khai trên Kubernetes, quản lý hạ tầng bằng Terraform, kết hợp với CI/CD, MLOps và DevSecOps.

II. YÊU CẦU BÀI TOÁN

a. Bối cảnh và mô tả bài toán

Hoạt động đọc thường tạo ra nhiều loại dữ liệu như thư viện cá nhân, tiến độ, nhận xét và lịch sử tương tác. Khi các dữ liệu này được ghi chép hoặc lưu trữ ở nhiều công cụ riêng lẻ, người dùng khó theo dõi quá trình đọc và kiểm soát nội dung đã chia sẻ. Vì vậy, Philobiblus được xây dựng như một hệ thống mẫu tập trung các thao tác quản lý thư viện, cập nhật tiến độ, nhận xét, chia sẻ và gợi ý sách.

Nếu mở rộng cho nhiều người dùng, hệ thống không chỉ cần đáp ứng các chức năng nghiệp vụ mà còn phải xử lý đồng thời nhiều request, phân tách dữ liệu theo tài khoản và kiểm soát phạm vi truy cập. Các thành phần frontend, backend, cơ sở dữ liệu và recommendation service được tách riêng để giới hạn ảnh hưởng khi có lỗi, triển khai độc lập và mở rộng theo tải. Đây là cơ sở để bài toán được triển khai trên Kubernetes với các yêu cầu về bảo mật, khả năng phục hồi, autoscale và theo dõi vận hành.

b. Mục tiêu của hệ thống

Mục tiêu nghiệp vụ của Philobiblus là hoàn thiện các chức năng cơ bản để quản lý thư viện, tiến độ đọc, nhận xét, chia sẻ và gợi ý sách; dữ liệu phải được gắn với đúng tài khoản và phạm vi truy cập.

Mục tiêu kỹ thuật là triển khai hệ thống nhiều thành phần trên GKE Autopilot bằng Terraform và Helm; xây dựng CI/CD để kiểm thử, quét bảo mật, build và phát hành image; quản lý snapshot, run, artifact và model version bằng DVC và MLflow. Phần vận hành cần kiểm chứng khả năng cô lập và khôi phục dịch vụ, autoscale theo tải và quan sát workload qua metric, log và trạng thái rollout.

c. Yêu cầu của hệ thống

Yêu cầu chức năng

Quản lý tài khoản và quyền truy cập. Người dùng có thể đăng ký, đăng nhập, cập nhật hồ sơ và đăng xuất; backend phải xác thực token, kiểm tra chủ sở hữu tài nguyên và áp dụng quyền quản trị ở phía máy chủ.

Quản lý thư viện và tiến độ đọc. Người dùng có thể tạo, tìm kiếm, chỉnh sửa và xóa sách thuộc quyền sở hữu; mỗi sách phải lưu được trạng thái dự định đọc, đang đọc hoặc đã hoàn thành, cùng tiến độ và lịch sử thay đổi.

Tương tác và chia sẻ. Người dùng có thể viết nhận xét, chấm điểm, theo dõi tài khoản khác

và chia sẻ sách theo các mức public, private hoặc restricted; dữ liệu không được hiển thị vượt quá visibility đã chọn.

Gợi ý nội dung. Hệ thống phải trả về danh sách sách liên quan từ catalog hợp lệ, loại bỏ sách nguồn và các bản ghi không đủ metadata, đồng thời vẫn phục vụ được thư viện khi dịch vụ gợi ý tạm thời không sẵn sàng.

Quản trị vòng đời model. Hệ thống phải tạo snapshot dữ liệu, ghi nhận run và metric, kiểm tra chất lượng candidate rồi chỉ phát hành model đạt điều kiện; model đang phục vụ phải truy nguyên được về snapshot, mã nguồn và artifact tương ứng.

Yêu cầu phi chức năng

Bảo mật và phân quyền. Mật khẩu phải được băm, API phải kiểm tra JWT và quyền sở hữu, secret không đặt trong mã nguồn; các lớp Gateway, Cloud Armor, NetworkPolicy và securityContext phải hạn chế truy cập từ ngoài vào trong cluster.

Khả năng triển khai lặp lại. Hạ tầng GKE, Cloud SQL, mạng và quyền truy cập phải được mô tả bằng Terraform để có thể kiểm tra thay đổi, tái tạo môi trường và tránh cấu hình thủ công không truy vết được. Đây là lý do dự án áp dụng Infrastructure as Code.

Khả năng mở rộng và tự phục hồi. Các thành phần chạy dạng container phải được đóng gói thành Deployment hoặc Job trên Kubernetes, có readiness/liveness probe, giới hạn tài nguyên và cơ chế rollout; nhờ đó workload có thể tự khởi động lại và mở rộng theo nhu cầu.

Tính lặp lại của phát hành. Mỗi thay đổi phải đi qua CI/CD gồm kiểm thử, kiểm tra dependency, quét bảo mật, build image và triển khai bằng Helm với image digest xác định; việc phát hành không phụ thuộc vào thao tác thủ công trên cluster.

Quản lý vòng đời dữ liệu và model. Snapshot, pipeline, metric, artifact và model version phải được lưu vết bằng DVC và MLflow, cho phép tái hiện một lần huấn luyện và rollback khi candidate không đạt quality gate. Đây là yêu cầu cốt lõi dẫn đến việc áp dụng MLOps.

Quan sát và xử lý sự cố. Hệ thống phải xuất metric, log và trạng thái rollout để Cloud Monitoring, Cloud Logging và Managed Service for Prometheus có thể phát hiện lỗi, theo dõi tải và hỗ trợ đối chiếu nguyên nhân.

Khả năng kiểm thử và bảo trì. Backend, recommendation service, Helm chart và Terraform module phải được kiểm thử độc lập; cấu trúc module, schema và hợp đồng API cần đủ rõ để thay đổi một thành phần không làm hỏng các thành phần còn lại.

d. Phạm vi bài toán và trách nhiệm pháp lý liên quan

Philobiblus là dự án độc lập do em thực hiện từ phân tích yêu cầu, phát triển giao diện, API và cơ sở dữ liệu đến MLOps, kiểm thử, đóng gói và triển khai. Sản phẩm được đánh giá qua các luồng quản lý thư viện, tiến độ, nhận xét, chia sẻ, gợi ý và khả năng duy trì ranh giới quyền hạn.

Dự án không xử lý thanh toán, giao nhận hoặc phân phối nội dung số. Hệ thống chỉ lưu trữ và hiển thị các bản ghi mô tả sách do người dùng cung cấp hoặc lấy từ nguồn phù hợp, gồm tên sách, tác giả, thể loại, thể, số trang và thông tin hoặc đường dẫn ảnh bìa. Hệ thống không lưu trữ, sao chép

hoặc cung cấp toàn văn, ebook, audiobook hay bản scan của sách. Việc sử dụng ảnh bìa trong môi trường thực tế phải tuân theo quyền và điều khoản của nguồn cung cấp; báo cáo này chỉ mô tả phạm vi kỹ thuật của hệ thống.

Hệ thống gợi ý sách sử dụng thông tin mô tả của từng cuốn sách trong cơ sở dữ liệu, chẳng hạn như tên sách, tác giả, thể loại và thể. Dữ liệu này được lưu thành snapshot để huấn luyện và đánh giá mô hình. Sau đó, mô hình xác định mức độ tương đồng giữa các cuốn sách và đề xuất những cuốn gần với sách người dùng đang xem hoặc đã chọn, trong phạm vi danh sách sách hiện có. Hệ thống gợi ý sách chỉ phục vụ mục đích kiểm chứng pipeline MLOps, chưa hướng tới xây dựng dịch vụ thương mại hoặc mô hình cá nhân hóa ở quy mô lớn.

III. LÝ THUYẾT, GIẢI PHÁP, THUẬT TOÁN VÀ QUÁ TRÌNH PHÁT TRIỂN HỆ THỐNG

a. Các lý thuyết, giải pháp và thuật toán liên quan

Các lý thuyết nền tảng

DevOps: là phương pháp kết nối hoạt động phát triển và vận hành thông qua một quy trình liên tục gồm viết mã, kiểm thử, đóng gói, triển khai và theo dõi hệ thống. Trong dự án, DevOps được thể hiện qua CI/CD, container, Infrastructure as Code, Helm và Kubernetes để giảm thao tác thủ công và bảo đảm mỗi phiên bản được triển khai theo cùng một quy trình [4].

DevSecOps mở rộng DevOps bằng cách đưa bảo mật vào từng bước của quy trình phát triển, thay vì chỉ kiểm tra trước khi đưa hệ thống lên production. Các nội dung kiểm tra gồm mã nguồn, dependency, container image, secret, cấu hình hạ tầng và quyền truy cập. Với Philobiblus, các bước kiểm tra này được tích hợp vào CI/CD để phát hiện sớm rủi ro trước khi artifact được phát hành [5].

CI/CD tự động hóa chuỗi kiểm thử, đóng gói và phát hành, giúp mỗi thay đổi được kiểm tra trước khi đi vào môi trường chạy [4, 6].

Infrastructure as Code biểu diễn hạ tầng bằng code có thể được quản lý bằng Git, giúp quản lý hạ tầng nhất quán và cho phép xem trước thay đổi trước khi áp dụng [7, 8].

Containerization giúp đóng gói ứng dụng cùng dependency trong image bất biến, giảm khác biệt giữa môi trường phát triển, kiểm thử và production [9].

Điều phối workload: Kubernetes quản lý Pod, Deployment, Service, probe, rollout và autoscaling để workload có thể tự phục hồi và mở rộng khi lượng người dùng biến động [10].

MLOps là tập hợp các thực hành kết hợp giữa phát triển mô hình ML và vận hành hệ thống nhằm tự động hóa, chuẩn hóa và quản lý vòng đời mô hình một cách hiệu quả [11].

Observability: là việc đo lường và theo dõi trạng thái bên trong của một hệ thống phần mềm dựa trên các dữ liệu đầu ra mà nó tạo ra trong quá trình hoạt động [12].

Các giải pháp và công cụ áp dụng

Terraform: là một công cụ giúp quản lý và tự động hóa hạ tầng công nghệ dưới dạng mã nguồn (IaC) thông qua các bước viết cấu hình, lập kế hoạch và áp dụng thay đổi [8].

Google Cloud và GKE: GKE Autopilot cung cấp cluster Kubernetes managed, còn Google Cloud cung cấp các dịch vụ cloud để deploy hệ thống vào hoạt động [13].

Docker tạo image cho các thành phần của hệ thống như frontend, backend, mô hình gợi ý,... để có thể deploy lên các môi trường khác như cloud.

Helm giúp người dùng dễ dàng định nghĩa, cài đặt, nâng cấp và quản lý các cấu hình phức tạp trên cụm Kubernetes thông qua các gói đóng gói sẵn gọi là Chart [14].

GitHub Actions: là nền tảng tự động hóa và CI/CD tích hợp trực tiếp ngay trên GitHub [6].

DVC (Data Version Control) là công cụ quản lý phiên bản dữ liệu và quy trình xử lý dữ liệu, hoạt động cùng với Git. [15].

MLflow là nền tảng theo dõi thí nghiệm và quản lý vòng đời mô hình học máy. Công cụ lưu thông tin về run, parameter, metric, artifact và model version [16].

Cloud SQL là dịch vụ cơ sở dữ liệu quan hệ được quản lý hoàn toàn trên Google Cloud, trong dự án sử dụng PostgreSQL để lưu dữ liệu nghiệp vụ [17].

Redis là máy chủ cấu trúc dữ liệu trong bộ nhớ, thường được dùng làm cache, queue hoặc lớp xử lý dữ liệu có yêu cầu phản hồi nhanh [18].

Cloud Monitoring là dịch vụ của Google Cloud dùng để thu thập, hiển thị và cảnh báo dựa trên các chỉ số vận hành của tài nguyên [19].

Managed Service for Prometheus: Managed Service for Prometheus là dịch vụ quản lý Prometheus trên Google Cloud, dùng để thu thập và truy vấn metric do các workload Kubernetes xuất ra [20, 21].

Các thuật toán sử dụng

TF-IDF biến metadata sách thành vector thưa, trong đó các từ xuất hiện nhiều trong một sách nhưng ít xuất hiện trong catalog có trọng số cao [22].

Cosine similarity là phương pháp đo góc giữa hai vector TF-IDF để xếp hạng các sách có nội dung gần với sách nguồn mà không phụ thuộc độ dài tuyệt đối của chuỗi [22].

Snapshot hash và quality gate: Hệ thống băm nội dung logic của snapshot, đối chiếu schema và mức suy giảm catalog trước khi cho phép candidate thay champion; cách kiểm soát này được triển khai riêng cho pipeline của Philobiblus và liên kết với dữ liệu, run cùng model version [15, 16].

b. Quá trình phát triển hệ thống

Hoàn thiện mã nguồn và artifact

Frontend được phát triển bằng React 18 và Vite. Giao diện tổ chức theo component-based architecture, sử dụng shadcn/ui với style base-nova, Tailwind CSS và Lucide icons; React Router quản lý điều hướng, còn các Context Provider tách trạng thái xác thực, chế độ xem và cài đặt. Các service module gọi REST API, nên frontend không thể truy cập trực tiếp vào database hoặc model artifact. Frontend được deploy trực tiếp lên GitHub Pages.

Backend dùng FastAPI và Uvicorn, tổ chức router theo domain như authentication, books, users,

reviews và reading history. SQLAlchemy quản lý model và session database, Pydantic schema kiểm tra dữ liệu vào ra, còn dependency injection cung cấp database session cho từng request. JWT, quyền sở hữu và visibility đối với thông tin sách được kiểm tra khi gọi API; middleware bổ sung CORS, metric và trace. Cuối cùng, em đóng gói toàn bộ thành docker image để triển khai lên Kubernetes.

Cơ sở dữ liệu và cache: PostgreSQL lưu dữ liệu nghiệp vụ theo schema được khai báo bằng SQLAlchemy; môi trường production sử dụng Cloud SQL thay cho database container trong GKE. Redis đảm nhiệm cache catalog và rate limit, giúp giảm số lần truy vấn database và giới hạn các request tốn tài nguyên. Backend cấu hình connection pool, timeout và cơ chế bỏ qua cache khi Redis tạm thời không khả dụng.

Recommendation service: chạy bằng FastAPI, sử dụng các metadata của sách như tên, tác giả, thể loại để gợi ý sách tương đồng. Pipeline tạo snapshot catalog, chuẩn hóa dữ liệu và dùng TF-IDF để huấn luyện model Joblib; DVC quản lý các bước xử lý, còn MLflow lưu metric và phiên bản model. Khi hoạt động, hệ thống xếp hạng sách bằng cosine similarity và đưa ra top 5 sách có cosine similarity cao nhất.

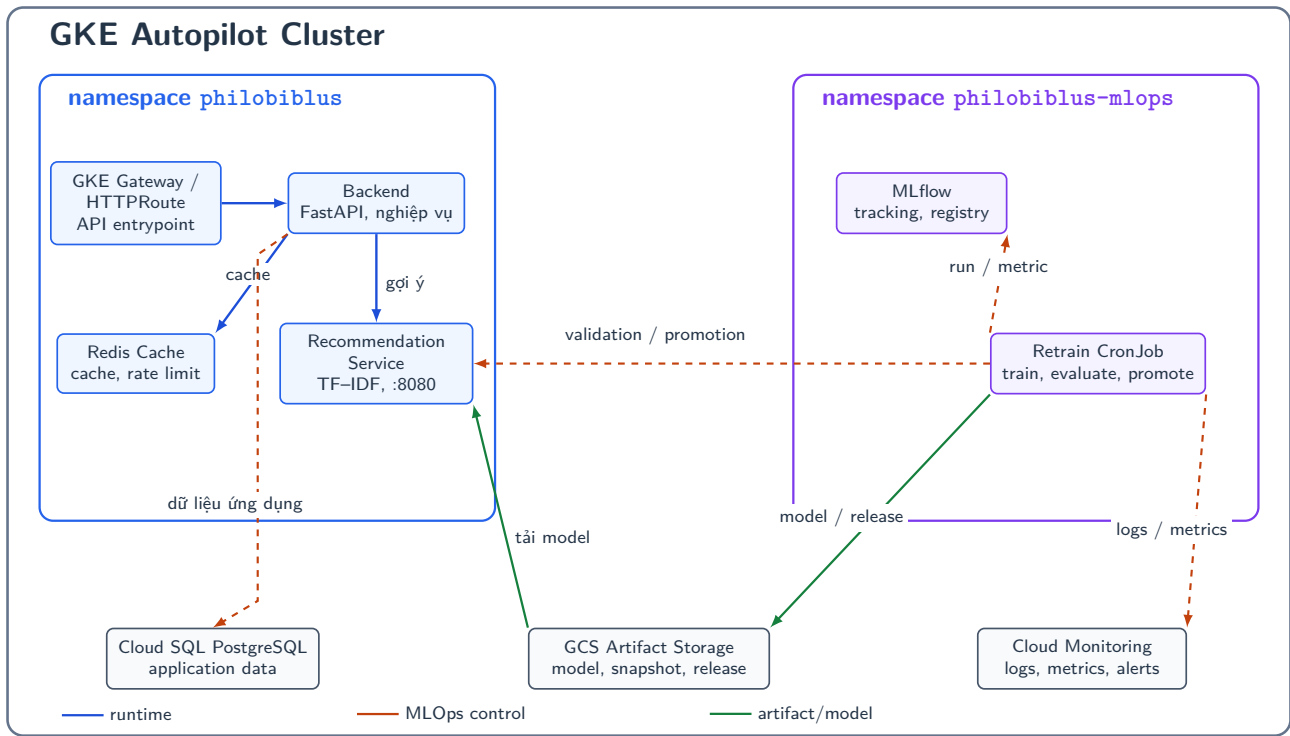
Tạo Helm chart

Hệ thống được tách thành 2 namespace: philobiblus chứa các workload phục vụ người dùng, còn philobiblus-mlops chứa các workload huấn luyện và phát hành model. Việc tách ra giúp hệ thống gợi ý sách có thể được phát triển và vận hành độc lập với các workload phục vụ người dùng, giảm rủi ro khi quá trình huấn luyện model gặp lỗi. Dưới đây là mô tả luồng hoạt động của hệ thống được thiết kế trong Helm chart:

Luồng runtime: Người dùng đi qua Gateway để vào nhóm workload ứng dụng. Gateway chuyển yêu cầu đến Backend; Backend đọc và ghi dữ liệu nghiệp vụ trong Cloud SQL, sử dụng Redis để lưu tạm dữ liệu thường dùng, rồi giao tiếp với Recommendation Service khi cần tạo gợi ý. Recommendation Service dùng model hiện hành đã được nạp trong Pod và trả kết quả về Backend để hoàn thành request.

Luồng MLOps control: Retrain CronJob trong namespace MLOps định kỳ lấy dữ liệu cần thiết, tạo snapshot, huấn luyện và đánh giá model. CronJob gửi run, metric và phiên bản model cho MLflow; trainer đồng thời kiểm tra Recommendation Service trước khi cho phép phát hành. Các workload gửi log và metric đến Cloud Monitoring để theo dõi rollout, lỗi và mức sử dụng tài nguyên. Các thao tác này được giới hạn bằng quyền của trainer, không làm namespace MLOps tham gia vào luồng request người dùng.

Luồng artifact/model: Sau khi model đạt điều kiện phát hành, trainer ghi model và thông tin release vào GCS. Recommendation Service lấy artifact tương ứng từ GCS, kiểm tra tính toàn vẹn rồi nạp model mới. Vì vậy, GCS đóng vai trò vùng lưu trữ trung gian giữa pipeline huấn luyện và workload phục vụ gợi ý.



Hình 1: Các workload và object chính trong cụm GKE của Philobibulus

Triển khai trên Google Cloud Kubernetes Engine

Helm chart sau khi được kiểm thử và xác minh, được deploy lên GKE Autopilot bằng Terraform. Luồng hoạt động trên Google Cloud gồm:

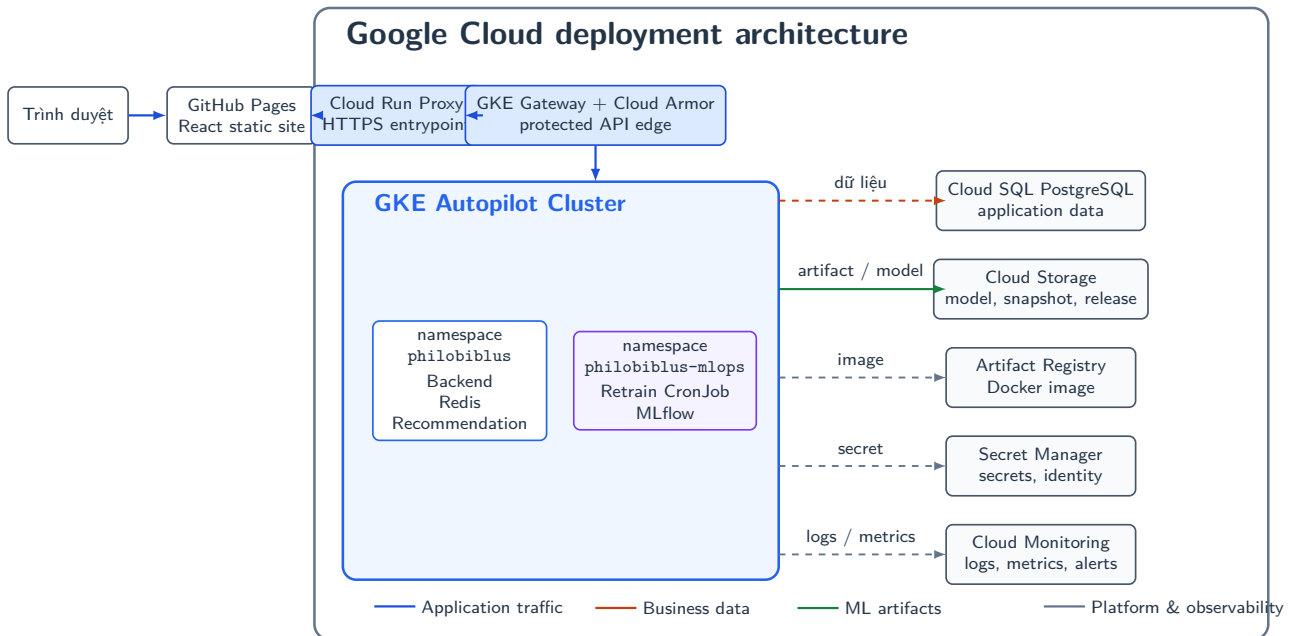
Application traffic: Người dùng truy cập frontend được phát hành trên GitHub Pages. Request đi qua Cloud Run Proxy rồi tới GKE Gateway và lớp bảo vệ Cloud Armor trước khi vào GKE Autopilot. Trong cluster, namespace ứng dụng tiếp nhận và xử lý request; namespace MLOps không nằm trên đường phục vụ người dùng.

Business data: Backend trong GKE đọc và ghi dữ liệu nghiệp vụ vào Cloud SQL PostgreSQL. Redis vẫn chạy trong namespace ứng dụng để lưu tạm dữ liệu thường dùng và giảm truy vấn lặp lại tới database. Cloud SQL được đặt ngoài cluster nên dữ liệu vẫn nằm trong dịch vụ cơ sở dữ liệu managed, còn workload Kubernetes chỉ sử dụng kết nối được cấp quyền.

ML artifacts: Retrain CronJob và MLflow trong namespace MLOps lưu snapshot, model và thông tin release vào Cloud Storage. Khi cần triển khai phiên bản mới, workload recommendation lấy artifact từ Cloud Storage để nạp model. Artifact Registry lưu các Docker image được build từ source; GKE sử dụng các image này khi tạo hoặc cập nhật workload.

Platform & observability: Secret Manager cung cấp secret và thông tin nhận dạng cho workload theo cơ chế được cấp quyền, thay vì đưa secret trực tiếp vào chart. Cloud Monitoring và Cloud Logging tiếp nhận log, metric và trạng thái vận hành từ cluster để theo dõi rollout, lỗi và mức sử dụng tài nguyên. Terraform giữ vai trò mô tả và tạo các tài nguyên này theo cùng một cấu hình có thể kiểm tra lại.

Tại thời điểm ngày 04/10/2026, cluster philobiblus-dev-gke đang chạy ổn định, tất cả các tài nguyên được deploy và hoạt động bình thường.



Hình 2: Luồng triển khai và các resource Google Cloud của Philobiblus

c. Ưu điểm và nhược điểm đang gặp phải

Ưu điểm. TF-IDF có chi phí tính toán thấp, dễ giải thích và vẫn có thể tạo gợi ý khi catalog chưa có nhiều lịch sử tương tác. Mô hình này phù hợp với giai đoạn đầu của Philobiblus vì dữ liệu sách và metadata đã có sẵn, trong khi dữ liệu hành vi người dùng còn hạn chế. Khi model được lưu trong MLflow và artifact được quản lý trên Cloud Storage, quá trình huấn luyện và phát hành có thể truy vết theo snapshot, run và phiên bản model.

Việc triển khai bằng Terraform, Helm và GKE giúp cấu hình hạ tầng và workload có thể kiểm tra, tái tạo và cập nhật theo cùng một quy trình. Backend có thể mở rộng bằng HPA, Redis giảm các truy vấn lặp lại, còn probe và rollout giúp hạn chế việc đưa Pod chưa sẵn sàng vào phục vụ. Cloud Monitoring và Cloud Logging cũng cung cấp dữ liệu để phát hiện lỗi và theo dõi mức sử dụng tài nguyên sau khi triển khai.

Nhược điểm và vấn đề đang gặp phải. Chất lượng gợi ý vẫn phụ thuộc mạnh vào metadata; TF-IDF chưa khai thác được mức độ tương đồng hình thành từ hành vi của cộng đồng người đọc. Các metric của run hiện tại còn có thể ở trạng thái *insufficient data*, vì vậy chưa nên diễn giải điểm đánh giá như một bằng chứng đầy đủ về chất lượng sản phẩm. Khi dữ liệu tương tác đủ lớn, hệ thống cần bổ sung mô hình kết hợp giữa đặc trưng nội dung và hành vi người dùng.

Redis chỉ giữ catalog trong thời gian ngắn và được làm mới khi dữ liệu thay đổi. Nếu Redis tạm thời không khả dụng, Backend bỏ qua cache để tiếp tục phục vụ nhưng sẽ tạo thêm tải cho Cloud SQL. Mỗi Backend Pod dùng pool ba kết nối cơ bản và tối đa thêm hai kết nối, với timeout mười giây; giới hạn này phải được tính cùng HPA để không vượt khả năng của Cloud SQL db-f1-micro.

GKE Autopilot, Cloud SQL, Cloud Storage, Secret Manager và các lớp bảo vệ mạng giúp hệ thống có cấu trúc rõ ràng hơn, nhưng đồng thời làm tăng số lượng tài nguyên phải quản lý, số quyền IAM cần kiểm soát và chi phí vận hành. Cloud Run Proxy bổ sung một lớp trung gian cho frontend, đổi lại tạo thêm một chặng trong luồng truy cập. Đây là các đánh đổi chấp nhận được cho môi trường thực tập và demo, nhưng cần được xem xét lại nếu hệ thống chuyển sang quy mô người dùng lớn hơn.

IV. CÁCH CÀI ĐẶT PHẦN MỀM

a. Triển khai phần mềm

Philobiblus được chia thành frontend, backend, cơ sở dữ liệu và các thành phần phục vụ hệ thống gợi ý. Frontend được phát hành riêng trên GitHub Pages, còn backend, recommendation service và các thành phần MLOps được triển khai trên cụm Kubernetes bằng Helm chart.

Frontend. Frontend được build và phát hành tự động bởi workflow [deploy-pages.yaml](#). Workflow này tạo bản build của giao diện React, đưa lên GitHub Pages và cấu hình địa chỉ API để frontend kết nối tới backend [6]. Vì frontend được cung cấp qua GitHub Pages nên không cần chạy thêm frontend Pod trên cụm Kubernetes.

Backend. Backend được đóng gói thành Docker Image [kazu912/philobiblus-backend](#) và triển khai thành các Pod trên GKE. Kubernetes cung cấp Service để nhận request từ frontend, đồng thời sử dụng Deployment, probe và Horizontal Pod Autoscaler để duy trì số lượng Pod phù hợp với tải hệ thống [9, 10, 13].

Cơ sở dữ liệu. Dữ liệu nghiệp vụ được lưu trên Cloud SQL for PostgreSQL, là dịch vụ cơ sở dữ liệu được Google Cloud quản lý. Backend kết nối tới Cloud SQL thông qua Cloud SQL Auth Proxy; thông tin xác thực được cung cấp bằng Secret thay vì lưu trực tiếp trong mã nguồn [17].

Recommendation service và MLOps. Recommendation service được đóng gói thành Docker Image [kazu912/philobiblus-recommendation](#) và phục vụ độc lập trên Kubernetes. Image [kazu912/philobiblus-model-fetcher](#) được dùng để tải model artifact vào môi trường chạy. Các thành phần huấn luyện và quản lý model được đóng gói lần lượt trong Docker Image [kazu912/philobiblus-trainer](#) và Docker Image [kazu912/philobiblus-mlflow](#).

Triển khai bằng Helm. Các thành phần ứng dụng được mô tả trong [Helm chart của ứng dụng](#), còn pipeline MLOps được mô tả trong [Helm chart MLOps](#). Helm cho phép cài đặt hoặc cập nhật nhiều tài nguyên Kubernetes từ cùng một bộ cấu hình [14].

Trên một cụm Kubernetes đã được cấu hình quyền truy cập, người triển khai chuẩn bị các giá trị môi trường, Secret và thông tin kết nối Cloud SQL, sau đó cài đặt Helm chart ứng dụng vào namespace `philobiblus`. Frontend được tắt trong chart vì đã được phát hành trên GitHub Pages. Helm chart MLOps được cài đặt riêng vào namespace `philobiblus-mlops` để tách các tác vụ huấn luyện và quản lý model khỏi workload phục vụ người dùng.

Sau khi clone repository, ta có thể cài đặt hai chart bằng các lệnh sau. Các file giá trị môi trường được giữ ngoài repository và chứa thông tin riêng của từng cụm Kubernetes.

```
helm upgrade --install philobiblus \\\
```

```
./kubernetes/helm/philobiblus \\  
--namespace philobiblus --create-namespace \\  
--set frontend.enabled=false -f <production-values.yaml>  
  
helm upgrade --install philobiblus-mlops \\  
./kubernetes/helm/philobiblus-mlops \\  
--namespace philobiblus-mlops --create-namespace \\  
-f <mlops-values.yaml>
```

Sau khi Helm hoàn tất triển khai, Kubernetes tạo các Deployment, Service, Secret reference, probe và autoscaling policy cần thiết cho từng thành phần. Khi cần cập nhật phần mềm, người triển khai thay đổi phiên bản các thành phần trong file giá trị môi trường rồi nâng cấp lại Helm release.

b. Về phía người dùng

Sau khi quá trình triển khai hoàn tất, Philobiblus là ứng dụng web nên người dùng chỉ cần mở địa chỉ GitHub Pages bằng trình duyệt, không phải cài đặt thêm phần mềm trên máy. Sau khi đăng nhập, người dùng có thể quản lý sách, cập nhật tiến độ đọc, viết nhận xét và nhận gợi ý từ recommendation service.

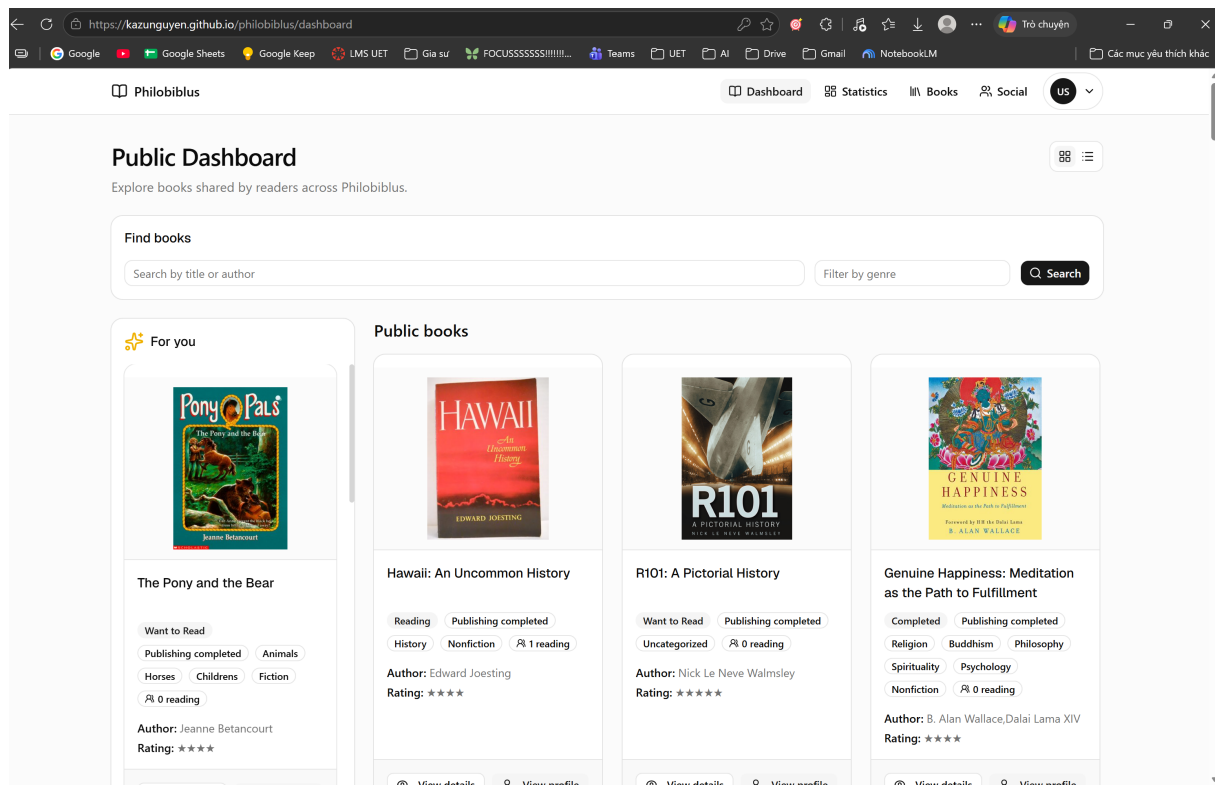
V. KẾT QUẢ ĐẠT ĐƯỢC VÀ HƯỚNG PHÁT TRIỂN TIẾP THEO

a. Kết quả đạt được

Sau khi hoàn thành quá trình phát triển và triển khai, Philobiblus đạt được ba nhóm kết quả chính: giao diện web có thể truy cập, pipeline recommendation được kết nối với dữ liệu và model, và hệ thống được vận hành trên GKE cùng các dịch vụ Google Cloud. Các minh chứng dưới đây được chụp từ phiên bản đang chạy của hệ thống.

Mã nguồn của hệ thống được lưu tại [repository Philobiblus trên GitHub](#). Phiên bản web dùng thử được phát hành tại [địa chỉ GitHub Pages của dự án](#).

Kết quả giao diện và gợi ý sách. Ở trang thư viện, khu vực *For you* sử dụng các đầu sách người dùng đã đọc làm tín hiệu đầu vào. Recommendation service phân tích metadata của những sách đó, sau đó trả về danh sách các sách có nội dung tương đồng để hiển thị trên giao diện.



Hình 3: Khu vực *For you* hiển thị gợi ý dựa trên lịch sử đọc của người dùng

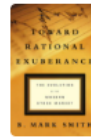
Khi người dùng mở trang chi tiết một cuốn sách, hệ thống dùng chính cuốn sách đang xem làm sách nguồn. Các metadata của sách nguồn được đưa vào bộ xếp hạng để tìm và hiển thị những đầu sách liên quan, giúp người dùng tiếp tục khám phá nội dung cùng chủ đề.

← Back

Toward Rational Exuberance: The Evolution of the Modern Stock Market

by B. Mark Smith

Shared by user1



Economics Finance History Personal Finance Nonfiction Currency Money Business

Reading status

Reading

Publication

Publishing completed

Rating

★★★★☆

Currently reading

👤 1 person

Started reading

8/8/2026

Notes

Demo catalog fixture from nice-bill/book-recommender-artifacts@e77f106e47c4430d0acdeef23b1cb096b31812a1; source rating: 3.73

Your reading progress

▶ Start reading

🌟 Recommended for this book

Content matched



The Single Best Investment: Creating Wealth with Dividend...

Reading Publishing completed

Economics Finance Business

Personal Finance Nonfiction

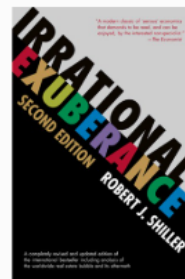
Currency Money 📖 1 reading

Author: Lowell Miller

Rating: ★★★★★

👁 View details

👤 View profile



Irrational Exuberance

Completed Publishing completed

Economics Finance Business

Nonfiction Psychology Currency

Money History 📖 0 reading

Author: Robert J. Shiller

Rating: ★★★★★

👁 View details

👤 View profile



The Divic Investme

Completer

Economics

Personal F

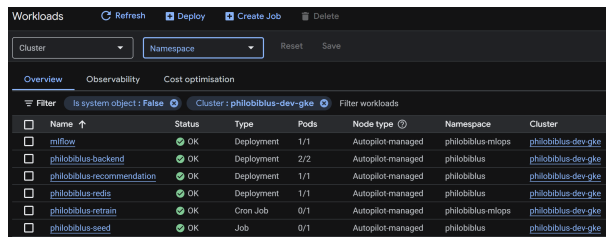
Author: R

Rating: ★1

👁 View

Hình 4: Gợi ý sách liên quan trên trang chi tiết của một cuốn sách

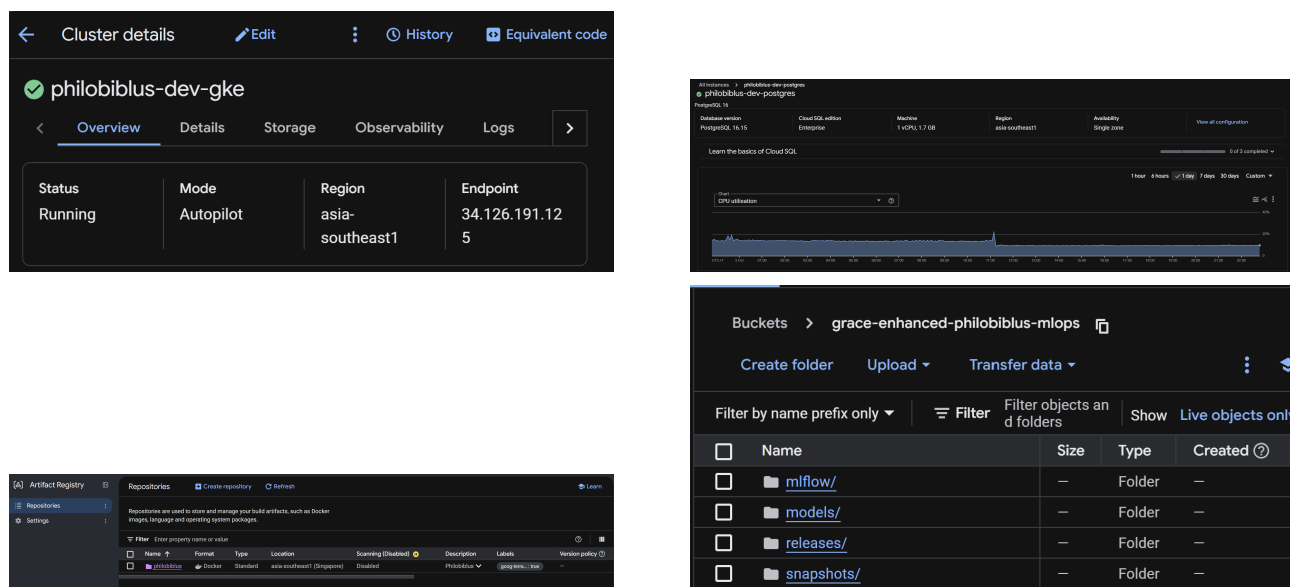
Kết quả triển khai và vận hành. Các workload phục vụ người dùng và MLOps được tách trong hai namespace của cùng một cụm GKE. Ảnh chụp dưới đây xác nhận các workload đã được tạo và đang được quan sát trên GKE Console.



Name	Status	Type	Pods	Node type	Namespace	Cluster
mlflow	OK	Deployment	1/1	Autopilot-managed	philobiblus-mlops	philobiblus-dev-gke
philobiblus-backend	OK	Deployment	2/2	Autopilot-managed	philobiblus	philobiblus-dev-gke
philobiblus-recommendation	OK	Deployment	1/1	Autopilot-managed	philobiblus	philobiblus-dev-gke
philobiblus-redis	OK	Deployment	1/1	Autopilot-managed	philobiblus	philobiblus-dev-gke
philobiblus-retain	OK	Cron Job	0/1	Autopilot-managed	philobiblus-mlops	philobiblus-dev-gke
philobiblus-seed	OK	Job	0/1	Autopilot-managed	philobiblus	philobiblus-dev-gke

Hình 5: Workload Philobiblus và MLOps quan sát trên GKE Console

Tại thời điểm ngày 04/10/2026, cluster philobiblus-dev-gke đang chạy ổn định, các tài nguyên chính phục vụ ứng dụng và MLOps đã được triển khai. Hình dưới đây đối chiếu trực tiếp trạng thái của GKE, Cloud SQL, Artifact Registry và Cloud Storage trên Google Cloud Console.



Hình 6: Trạng thái hoạt động của các tài nguyên Google Cloud: GKE, Cloud SQL, Artifact Registry và Cloud Storage

Kiểm thử và xác minh

Các ca kiểm thử được thực hiện trên trạng thái source hiện tại ngày 04/10/2026. Backend và ML chạy trong môi trường Python tạm với dependency lấy từ file requirements; build frontend dùng fixture tương đương GitHub Pages; Helm dùng fixture không chứa secret như workflow CI.

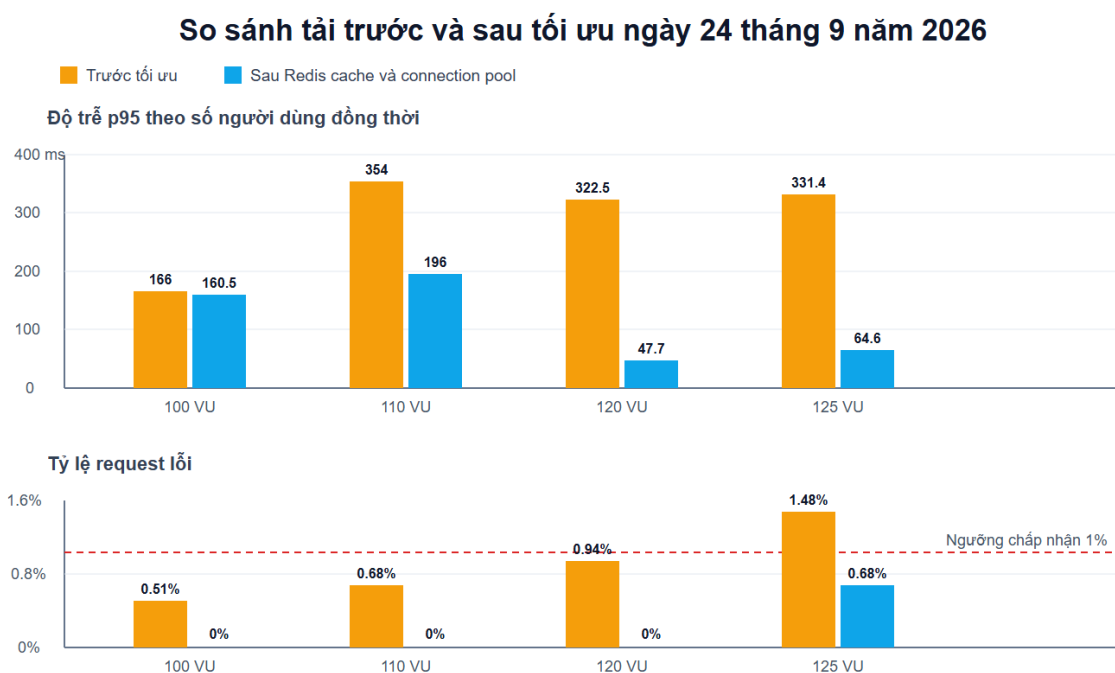
Mã	Cá kiểm thử và phạm vi	Kết quả
TC-01	Backend pytest: API nghiệp vụ, JWT và phân quyền, cache catalog, rate limit, observability, recommendation và load shedding.	63/63 đạt trong 60,03 giây; 2 cảnh báo deprecation từ dependency.
TC-02	ML pytest: recommendation service, TF-IDF recommender, quality gate và snapshot validation.	17/17 đạt trong 6,89 giây; 1 cảnh báo deprecation từ Starlette.
TC-03	Frontend build Vite với biến môi trường tương đương workflow GitHub Pages.	Thành công: 2.105 module được bundle trong 8,27 giây.
TC-04	Lint và render chart philobiblus và philobiblus-mlops với fixture không chứa secret.	2/2 chart thành công; chỉ có khuyến nghị thêm icon cho Chart.yaml.
TC-05	Truy vấn chỉ đọc trạng thái GKE, Cloud Run, Cloud SQL, Artifact Registry và Cloud Storage.	GKE RUNNING, proxy READY, Cloud SQL RUNNABLE; repository và bucket MLOps tồn tại.

Table 1: Kết quả kiểm thử và xác minh triển khai

Backend pytest dùng SQLite in-memory để cô lập test API, nên không thay thế kiểm thử tích hợp với Cloud SQL. Helm lint và template kiểm tra tính hợp lệ của manifest trước triển khai, không thay thế việc rollout trên cluster. Hai giới hạn này được giữ tách biệt với kết quả xác minh resource GCP để tránh diễn giải quá mức phạm vi kiểm thử.

Kiểm thử tải

Lần kiểm tra đầu đạt 120 người dùng đồng thời với 23,4 request/giây, p95 322,5 ms và 0,94 phần trăm lỗi; tại 125 người dùng, lỗi tăng lên 1,48 phần trăm. Sau khi thêm Redis cache TTL 30 giây và giới hạn pool 3 + overflow 2 cho mỗi Pod, mốc 150 người dùng đạt 29,6 request/giây, p95 138,6 ms và không có request lỗi.



Hình 7: Độ trễ và tỷ lệ lỗi tại các mức tải chung trước và sau tối ưu

150 người dùng là mức cao nhất đã kiểm tra; ở lần benchmark này recommendation service được tắt để cô lập backend cùng database. Vì vậy kết quả không đại diện cho năng lực khi đồng thời phục vụ workload gợi ý. Môi trường dev hiện bật recommendation service và dùng circuit breaker Redis chung: vượt 12 request trong 10 giây thì API trả unavailable trong 30 giây, sau đó tự phục hồi. Mức vận hành đề xuất vẫn là 120 đến 130 người dùng đồng thời, khoảng 24 request/giây, trước khi thực hiện lần đo đầy đủ hơn có recommendation service.

Đối chiếu với các yêu cầu tại Chương II, Philobiblus đã đáp ứng các luồng chức năng cốt lõi gồm quản lý tài khoản, thư viện, tiến độ đọc, tương tác, chia sẻ và gợi ý sách. Các yêu cầu phi chức năng về bảo mật, triển khai lặp lại, mở rộng, MLOps, quan sát và kiểm thử đã được áp dụng ở mức phù hợp với phạm vi thực tập; một số giới hạn còn lại nằm ở chất lượng mô hình, kiểm thử tích hợp, rollback và quy trình CD production.

b. Kỹ năng và kiến thức thu thập được

Trong kỳ thực tập, em đã được thực hành theo dõi toàn bộ hệ thống từ mã nguồn, kiểm thử và image đến Terraform, Helm, GKE và quan sát vận hành, nhờ đó em có thể hiểu rõ hơn cách Service, probe, resource, secret, policy mạng và rollout phối hợp trong một workload thực tế. Ngoài ra, nhờ được thực hành quản lý snapshot, artifact, quality gate, JWT, quyền sở hữu, Cloud Armor, NetworkPolicy và kiểm tra chuỗi cung ứng phần mềm, em đã củng cố kiến thức về DevOps, MLOps và DevSecOps, đồng thời rèn luyện kỹ năng lập trình, kiểm thử, triển khai và vận hành hệ thống cho mình.

d. Hướng phát triển tiếp theo để hoàn thiện giải pháp

Kết quả benchmark cho em thấy khá rõ giới hạn của cấu hình hiện tại: hệ thống phù hợp hơn

với khoảng 120 đến 130 người dùng đồng thời, còn mốc 150 người dùng mới chỉ được đo khi tắt recommendation service. Nếu tiếp tục phát triển, em sẽ phải đánh giá lại toàn bộ luồng, bao gồm cả nghiệp vụ và recommendation, thay vì chỉ nhìn vào kết quả benchmark đã cô lập backend với cơ sở dữ liệu. Những giới hạn về cache, connection pool và năng lực Cloud SQL sẽ là dữ liệu đầu vào để lựa chọn cách phân vùng tải và chính sách autoscaling ở các giai đoạn sau.

Ở giai đoạn sau, hệ thống sẽ cần chuyển Cloud Armor từ chế độ preview sang enforce, xác minh nguồn gốc artifact bằng chữ ký và nổi trace, PolicyReport, log với cảnh báo trong cùng một quy trình xử lý sự cố. Còn đối với chức năng gợi ý sách, em muốn tiếp tục thu thập thêm lịch sử đọc để đánh giá mô hình trên dữ liệu thực tế hơn. Sau khi có đủ dữ liệu, em sẽ train model gợi ý mới kết hợp thông tin nội dung với hành vi người dùng để cải thiện chất lượng gợi ý.

TÀI LIỆU THAM KHẢO

- [1] JITS Innovation Labs. (2026) Jits innovation labs. [Online]. Available: <https://jitsinnovationlabs.com/>
- [2] ——. (2026) Jits innovation labs trên linkedin. [Online]. Available: <https://www.linkedin.com/company/jits-innovation-labs>
- [3] Cổng thông tin quốc gia về đăng ký doanh nghiệp. (2026) Thông tin đăng ký doanh nghiệp công khai. [Online]. Available: <https://dangkykinhdoanh.gov.vn/>
- [4] Amazon Web Services. (2026) What is devops? [Online]. Available: <https://aws.amazon.com/devops/what-is-devops/>
- [5] ——. (2026) What is devsecops? [Online]. Available: <https://aws.amazon.com/what-is/devsecops/>
- [6] GitHub. (2026) Github actions documentation. [Online]. Available: <https://docs.github.com/en/actions>
- [7] HashiCorp. (2026) What is infrastructure as code with terraform? [Online]. Available: <https://developer.hashicorp.com/terraform/tutorials/aws-get-started/infrastructure-as-code>
- [8] ——. (2026) Overview of the core terraform workflow. [Online]. Available: <https://developer.hashicorp.com/terraform/intro/core-workflow>
- [9] Docker, Inc. (2026) What is a container? [Online]. Available: <https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>
- [10] The Kubernetes Authors. (2026) Deployments. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/>
- [11] Google Cloud. (2024) Mlops: Continuous delivery and automation pipelines in machine learning. [Online]. Available: <https://docs.cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [12] OpenTelemetry Authors. (2026) Observability primer. [Online]. Available: <https://opentelemetry.io/docs/concepts/observability-primer/>

- [13] Google Cloud. (2026) Gke autopilot overview. [Online]. Available: <https://docs.cloud.google.com/kubernetes-engine/docs/concepts/autopilot-overview>
- [14] Helm Authors. (2026) Charts. [Online]. Available: <https://helm.sh/docs/topics/charts/>
- [15] Iterative, Inc. (2026) Dvc command reference. [Online]. Available: <https://dvc.org/doc/command-reference>
- [16] MLflow Authors. (2026) MLflow tracking. [Online]. Available: <https://mlflow.org/docs/latest/ml/tracking>
- [17] Google Cloud. (2026) Cloud sql overview. [Online]. Available: <https://docs.cloud.google.com/sql/docs/postgres/introduction>
- [18] Redis. (2026) Redis data types. [Online]. Available: <https://redis.io/docs/latest/develop/data-types/>
- [19] Google Cloud. (2026) Cloud monitoring overview. [Online]. Available: <https://docs.cloud.google.com/monitoring/docs/monitoring-overview>
- [20] Prometheus Authors. (2026) Metric types. [Online]. Available: https://prometheus.io/docs/concepts/metric_types/
- [21] Google Cloud. (2026) Managed service for prometheus. [Online]. Available: <https://docs.cloud.google.com/stackdriver/docs/managed-prometheus>
- [22] Scikit-learn Developers. (2026) Tfidfvectorizer and cosine similarity. [Online]. Available: <https://scikit-learn.org/stable/>



Ý kiến đánh giá:

..... - Có tinh thần tìm hiểu, ham học hỏi.....
..... Hoàn thành đúng hạn các nhiệm vụ được giao.....
..... Thái độ và tác phong làm việc nghiêm túc.....
.....
.....
.....
.....
.....

Hà Nội, ngày 10 tháng 08 năm 2026 .

Người hướng dẫn

(Ký, ghi rõ họ tên & dấu công ty)

Thanh

Bùi Tiến Thanh



Ý kiến đánh giá:

Điểm số:

Điểm chữ:

Hà Nội, ngày tháng năm 20 .

Giảng viên đánh giá

(Ký, ghi rõ họ tên)